

Южно-Уральский государственный университет

Национальный исследовательский университет



НТЦ ПОЛИТЕХНИКА

технологии • решения • интеграция

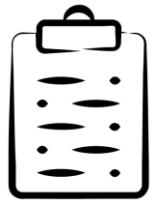
3.2. JavaScript. Стрелочные функции. Fetch API

2026 ООО НТЦ «Политехника»
Челябинск, ул. Карла Маркса, д. 38

Кузнецов Сергей

Фоменко Алексей

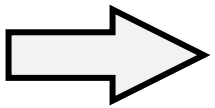
Стрелочные функции



- Зачастую конструкция function() может занимать много места, в ES6 ввели новое объявление функций в стиле «лямба-выражений»

```
(param1, param2, …, paramN) => {
    statements
}
```

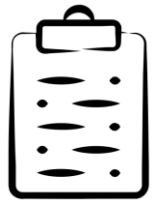
```
function f(a1, a2) {
    console.log(a1 + a2);
}
```



```
const f = (a1, a2) => {
    console.log(a1 + a2);
}
```

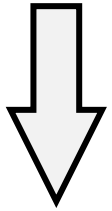
Стрелочные функции

НТЦ ПОЛИТЕХНИКА



- Если возвращаемое значение короткое, то можно упразднить {}:

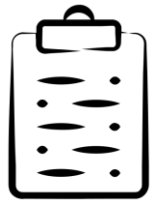
```
(param1, param2, …, paramN) =>
{
    return expression;
}
```



```
(param1, param2, …, paramN) => expression
```

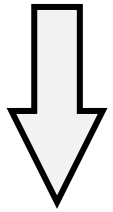
Стрелочные функции. Пример 1

НТЦ ПОЛИТЕХНИКА

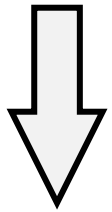


- Пример объявления функции без фигурных скобок:

```
function f(a1, a2) {
    console.log(a1 + a2);
}
```



```
const f = (a1, a2) => {
    console.log(a1 + a2);
}
```

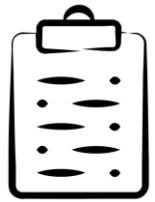


```
const f = (a1, a2) => console.log(a1 + a2);
```

```
> const f = (a1, a2) => console.log(a1 + a2);
< undefined
> f(1, 2)
  3
```

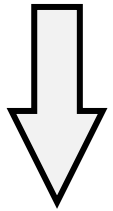
Стрелочные функции. Пример 2

НТЦ ПОЛИТЕХНИКА

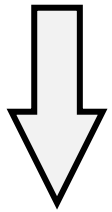


- Пример объявления функции без фигурных скобок:

```
function f (x, y) {
    return x * y;
}
```



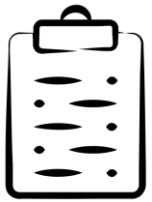
```
const f = (x, y) => {
    return x * y;
}
```



```
const f = (x) => x * x;
```

Стрелочные функции с многострочным телом

НТЦ ПОЛИТЕХНИКА



- Фигурные скобки нужны, если тело функции требует нескольких операций или для повышения читаемости:

```
const f = (a1, a2) => {
    if (a1 > 0) {
        console.warn(a1 + a2);
    }

    const s = `Текст ${a1} и ${a2}`;
    return s;
}

const result = f(1, 2);
```

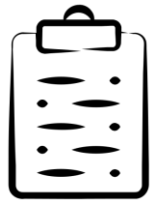
```
> const f = (a1, a2) => {
      if (a1 > 0) {
          console.warn(a1 + a2);
      }

      const s = `Текст ${a1} и ${a2}`;
      return s;
  }

  const result = f(1, 2);
▲ ▶ 3
  undefined
> result
  'Текст 1 и 2'
```

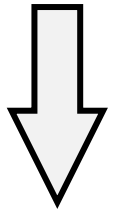
Стрелочные функции без аргумента

НТЦ ПОЛИТЕХНИКА



- Если аргументов нет, то при объявлении должны быть просто пустые скобки ():

```
function f() {
    console.log(123);
}
```

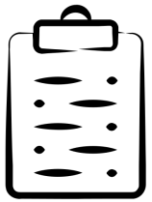


```
const f = () => console.log(123);
```

```
> const f = () => console.log(123);
<· undefined
> f()
  123
```

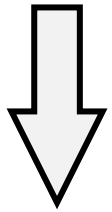
Стрелочные функции с одним аргументом

НТЦ ПОЛИТЕХНИКА

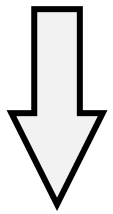


- Если аргумент один, то круглые скобки можно упразднить:

```
function f(a1) {
    console.log(a1);
}
```



```
const f = (a1) => console.log(a1);
```

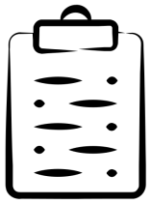


```
const f = a1 => console.log(a1);
```

```
> const f = a1 => console.log(a1);
undefined
> f('Hello')
Hello
```

Пример использования стрелочных функций

НТЦ ПОЛИТЕХНИКА



- Аналогичные по функционалу конструкции:

```
[1, 2, 3].forEach(function(item) {
    console.log(item * 2)
})

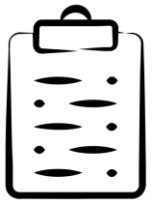

[1, 2, 3].forEach((item) => {
    console.log(item * 2)
})


[1, 2, 3].forEach(item => {
    console.log(item * 2)
})


[1, 2, 3].forEach(item => console.log(item * 2));
```

Контекст вызова в () => и в function

НТЦ ПОЛИТЕХНИКА

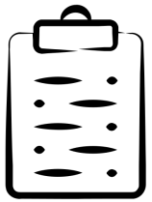


- Основное отличие в контексте вызова this
- Стрелочная функция не имеет своего контекста и берет его из внешней области

```
> const xx = {
      key: 123,
      f1: function() {
          return this;
      },
      f2: () => {
          return this;
      }
  }
< undefined
> xx.f1()
<   ▶ {key: 123, f1: ƒ, f2: ƒ}
> xx.f2()
<   ▶ Window {0: Window, window: Window, self: Window, document: document, name
```

Контекст вызова в () => и в function

НТЦ ПОЛИТЕХНИКА



Если же создавать стрелочные функции как методы класса, то контекст вызова в таком случае сохраняется и можно получить через this доступ к полям.

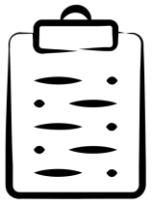
```
> const xx = {
      key: 123,
      f2: () => {
          return this.key; // undefined
      }
  }
  xx.f2()
< undefined
```

```
> class SomeClass {
      key = 123;
      f2 = () => {
          return this.key;
      };
  }

  const yy = new SomeClass();
  yy.f2()
< 123
```

Аналогии с C#

НТЦ ПОЛИТЕХНИКА



- Аналогичные по функционалу конструкции:

C#

```
var numbers = new List<int>
{
    1, 2, 3, 4, 5, 6, 7, 8, 9, 10
};

numbers
    .Where(n => n > 5)
    .Select(n => n * 2)
    .OrderByDescending(a => a)
    .ToList()
    .ForEach(n => Console.WriteLine(n));
```

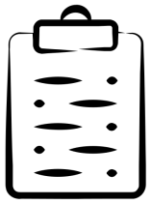
JS

```
const numbers =
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

numbers
    .filter(n => n > 5)
    .map(n => n * 2)
    .sort((a, b) => b - a)
    .forEach(n => console.log(n));
```

Вывод: 20 18 16 14 12

Аналоги с C#



В C# также можно задавать методы класса стрелочными функциями.

C#

```
class Program
{
    class SomeClass
    {
        private readonly int key = 123;
        public int F2() => key;
    }

    static void Main()
    {
        var yy = new SomeClass();
        Console.WriteLine(yy.F2());
    }
}
```

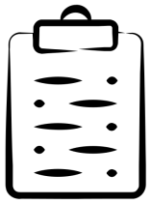
JS

```
class SomeClass {
    key = 123;
    f2 = () => this.key;
}

const yy = new SomeClass();
yy.f2()
```

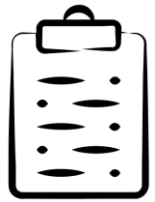
Запросы к серверу со стороны JS

НТЦ ПОЛИТЕХНИКА



- Очень распространённой задачей в современных веб-сайтах и приложениях является получение отдельных элементов данных с сервера для обновления разделов веб-страницы без необходимости загрузки всей новой страницы
- JavaScript может отправлять сетевые запросы на сервер и подгружать новую информацию по мере необходимости

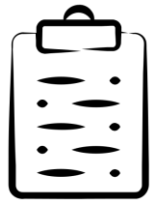
JS. fetch



- Fetch API предоставляет интерфейс JavaScript для работы с запросами и ответами HTTP.
- Он также предоставляет глобальный метод fetch()

```
fetch('http://example.com', {
  // Опции
})
.then(function(response) {
  return response.json();
})
.then(function(data) {
  console.log(data);
});
```

JS. fetch. Response. Свойства



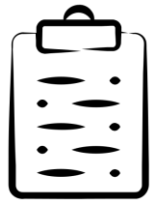
Для представления ответа от сервера в Fetch API применяется интерфейс Response. Свойства:

- ok: хранит булевое значение, которое указывает, завершился ли запрос
- status: хранит статусный код ответа
- statusText: хранит сообщение статуса, которое соответствует статусному коду
- …

```
fetch('http://example.com')
  .then(function(response) {
    console.log(response)
  })
```

```
▼ Response {type: 'basic', url: 'http://loca
  ▶ body: ReadableStream
    bodyUsed: true
  ▶ headers: Headers {}
    ok: true
    redirected: false
    status: 200
    statusText: "OK"
    type: "basic"
    url: "http://localhost/render/api.php"
```

JS. fetch. Response. Методы



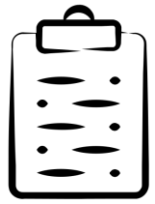
Методы:

- text() для текста
- json() для текста в формате JSON
- blob() для бинарных объектов (изображения, аудио и т.д.)
- …

```
fetch('http://example.com')
  .then(function(response) {
    return response.json();
  })
  .then(function(data) {
    // data – объект из JSON
  });
```

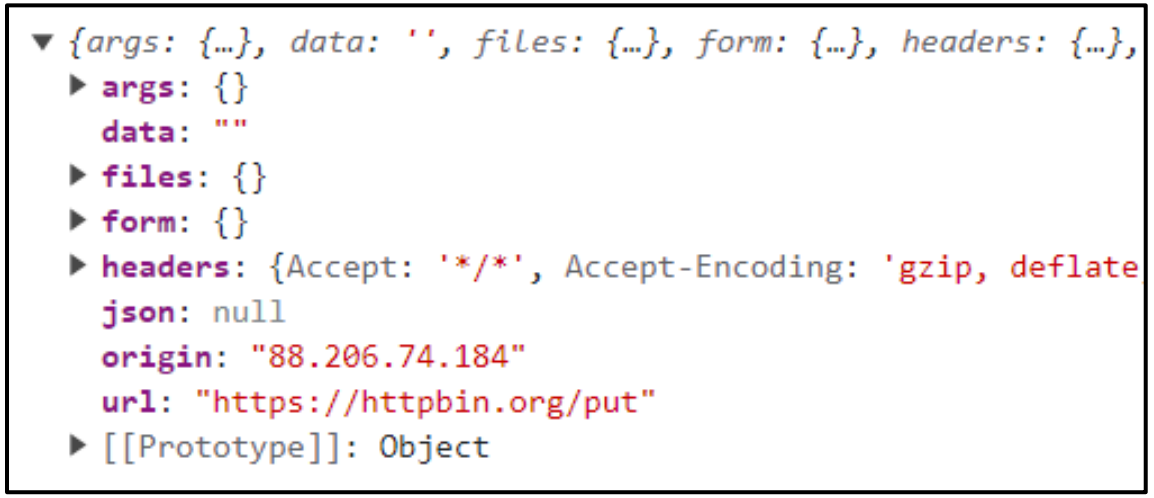
JS. fetch. Метод HTTP

НТЦ ПОЛИТЕХНИКА

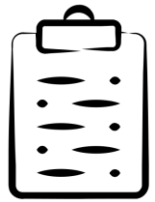


Для установки метода используется свойство method в опциях fetch. По умолчанию GET

```
fetch('https://httpbin.org/put', {
  method: 'PUT'
})
.then(function(response) {
  return response.json()
})
.then(function(data) {
  console.log(data)
})
```



JS. fetch. Тело запроса



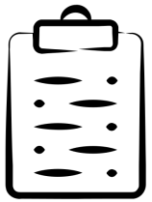
Для отправки тела используется свойство body в опциях fetch

```
let data = {
  key1: 'value1'
}
fetch('https://httpbin.org/post', {
  method: 'POST',
  body: JSON.stringify(data)
})
.then(function(response) {
  return response.json()
})
.then(function(data) {
  console.log(data)
})
```



JS. fetch. Заголовки

НТЦ ПОЛИТЕХНИКА



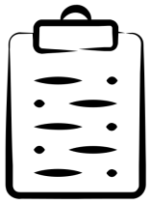
Для установки заголовков используется свойство headers в опциях fetch

```
let data = {
  key1: 'value1'
}
fetch('https://httpbin.org/post', {
  method: 'POST',
  headers: {
      'Content-Type': 'application/json'
  },
  body: JSON.stringify(data)
})
.then(function(response) {
  return response.json()
})
.then(function(data) {
  console.log(data)
})
```



JS. fetch. Обработка ошибок

НТЦ ПОЛИТЕХНИКА



Для обработки ошибок применяется метод catch

```
fetch('https://httpbin.org/post', { // post
  method: 'PUT' // put
})
.then(function(response) {
  console.log('Статус: ' + response.status)
  return response.json()
})
.then(function(data) {
  console.log(data)
})
.catch(function(error) { // Обработка ошибок
  console.warn('Ошибка!')
  console.warn(error.message)
});
```

```
Статус: 405
▶ Ошибка!
▶ Unexpected token '<', "<!DOCTYPE "...
```

JS. fetch. Обработка ошибок

НТЦ ПОЛИТЕХНИКА

```
fetch('https://httpbin.org/post', { // post
  method: 'PUT' // put
})
.then(function(response) {
  console.log('Статус: ' + response.status)
  if (response.status === 200) {
    return response.json()
  }
  return response.text()
})
.then(function(data) {
  console.log(data)
})
.catch(function(error) {
  console.warn('Ошибка!')
  console.warn(error.message)
});
```

```
Статус: 405
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>
```

JS. fetch. Стрелочные функции

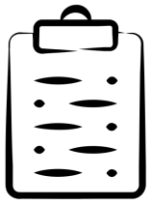
НТЦ ПОЛИТЕХНИКА

```
fetch('https://httpbin.org/post', { // post
  method: 'PUT' // put
})
.then(response => {
  console.log('Статус: ' + response.status)
  if (response.status === 200) {
    return response.json()
  }
  return response.text()
})
.then(data => console.log(data))
.catch(error => {
  console.warn('Ошибка!')
  console.warn(error.message)
});
```

```
Статус: 405
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>
```

Асинхронное программирование async/await

НТЦ ПОЛИТЕХНИКА



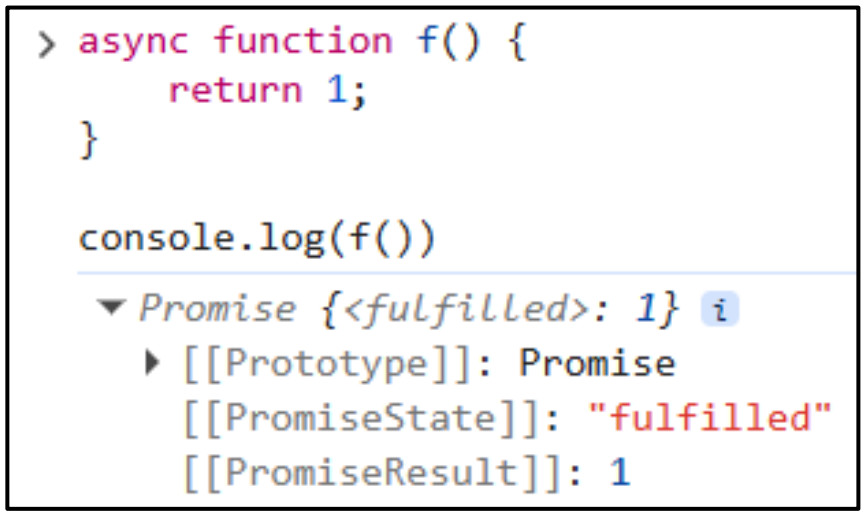
В стандарте ES 2017 появился специальный синтаксис для работы с промисами

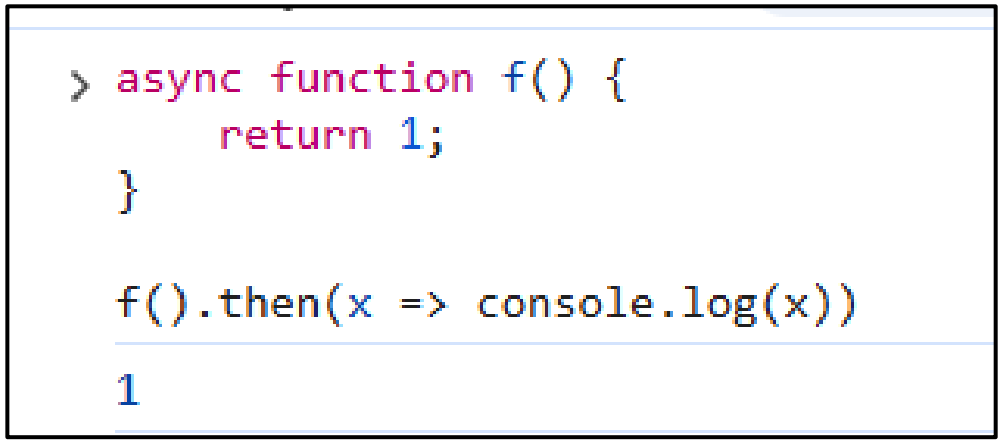
```
async function f() {
    return 1;
}

console.log(f())
```

```
async function f() {
    return 1;
}

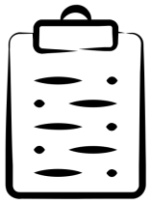
f().then(x => console.log(x))
```





Асинхронное программирование async/await

НТЦ ПОЛИТЕХНИКА



- Ключевое слово async перед функцией гарантирует, что эта функция в любом случае вернёт промис
- Ключевое слово await заставит интерпретатор JavaScript ждать до тех пор, пока промис справа от await не выполнится

```
async function f() {
    return 1;
}

const result = await f()
console.log(result)
```

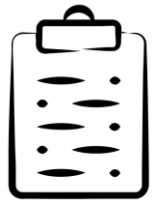
```
> async function f() {
      return 1;
  }

  const result = await f()
  console.log(result)

  1
```

Синтаксис Fetch с async/await

НТЦ ПОЛИТЕХНИКА

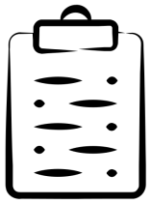


- Для fetch удобно использовать подход с async и await, так как код становится похож на обычный синхронный

```
async function getResult() {
    try {
        const response = await fetch('https://httpbin.org/post');
        return await response.json();
    } catch(error) {
        console.warn('Ошибка!')
        console.warn(error.message)
    }
}
```

Плюсы и минусы async/await в JS

НТЦ ПОЛИТЕХНИКА



Плюсы:

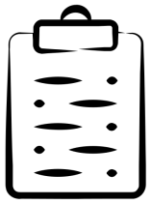
- Синтаксис проще и чище, код похож на синхронный, т.к. нет функций обратного вызова и лишней вложенности
- Можно использовать с обычными try/catch

Минусы :

- Работает только внутри async функций, нельзя вызвать await в обычной функции
- Помимо fetch остальные области применения async в клиентском JS не так распространены (мультимедия, анимации)

Сравнение async/await с JS

НТЦ ПОЛИТЕХНИКА



- JavaScript **однопоточный**, все асинхронные операции на самом деле выполняются в одном потоке. Поэтому эффективности по ресурсам такие конструкции не прибавляют, только улучшение синтаксиса.
- В C# ситуация другая – он **многопоточный**. Там происходит экономия оперативной памяти и потоков из-за использования асинхронного подхода и в нагруженных приложениях (особенно серверы) имеется значительный прирост производительности.

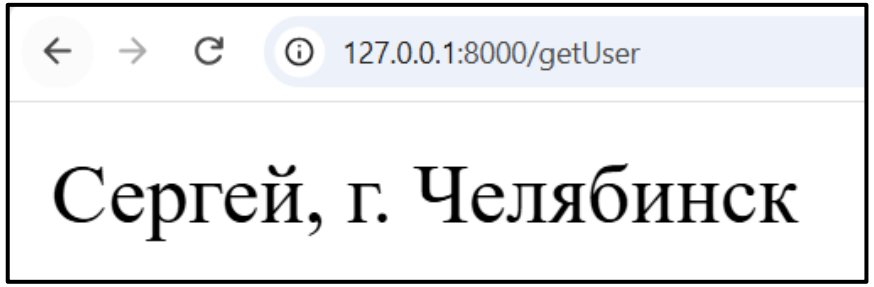
Серверный рендеринг HTML

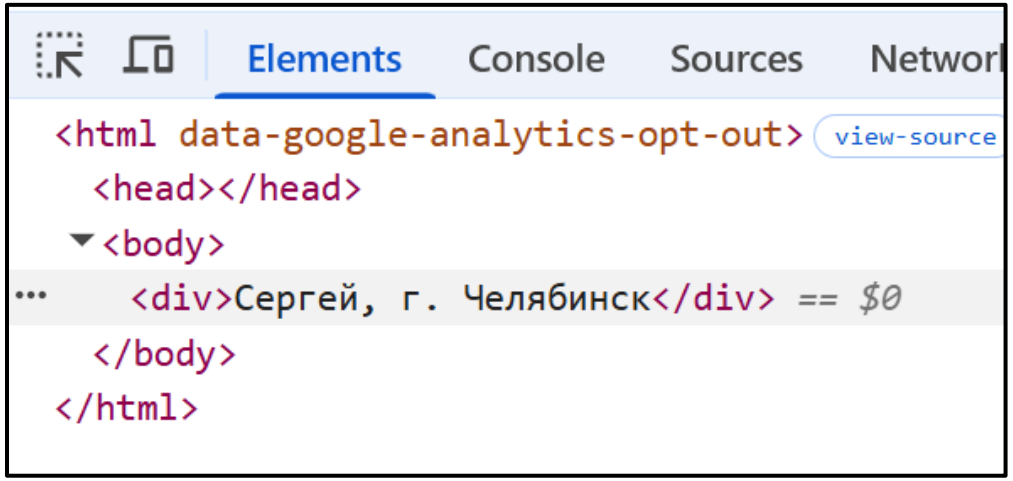
НТЦ ПОЛИТЕХНИКА

```python
from fastapi import FastAPI
from fastapi.responses import HTMLResponse

app = FastAPI()
data = {"name": "Сергей", "city": "Челябинск"}

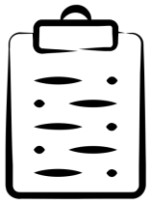
@app.get("/getUser", response_class=HTMLResponse)
def home():
    return f"<div>{data['name']}, г. {data['city']}</div>"
```





Серверный рендеринг HTML

НТЦ ПОЛИТЕХНИКА

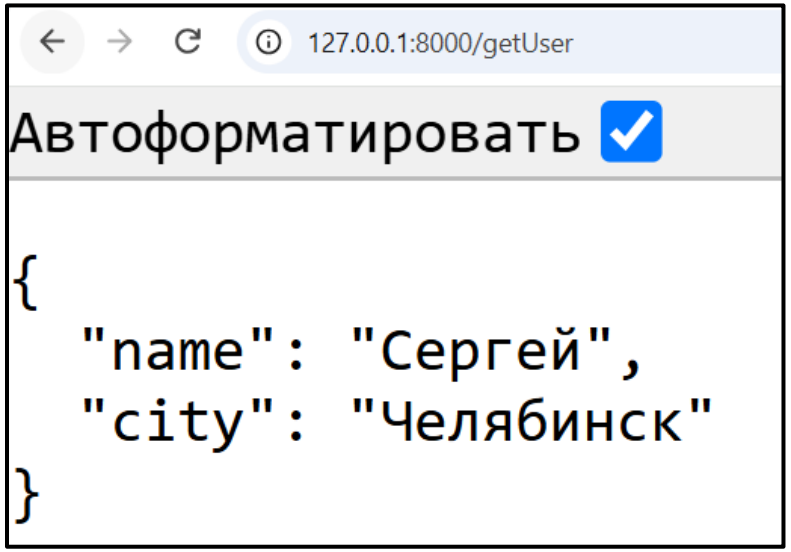


- Меняем API сервера, чтобы он возвращал JSON

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

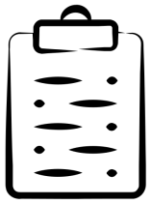
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"]
)
data = {"name": "Сергей", "city": "Челябинск"}

@app.get("/getUser")
def home():
    return data
```



Клиентский рендеринг

НТЦ ПОЛИТЕХНИКА



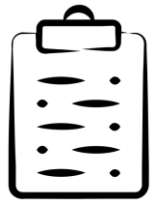
- Обращаемся из клиента к серверу

```
<meta charset="UTF-8">
<button id="sendButton" onclick="getUsers()">Получить</button>
<div id="container"></div>
<script>
const url = 'http://127.0.0.1:8000/getUser';
const cont = document.querySelector('#container');
function onSuccess(data) {
    cont.innerHTML =
        `<div>${data.name}, г. ${data.city}</div>`;
}
const getUsers = async () =>  {
      const result = await fetch(url);
      const data = await result.json();
      onSuccess(data);
}
</script>
```

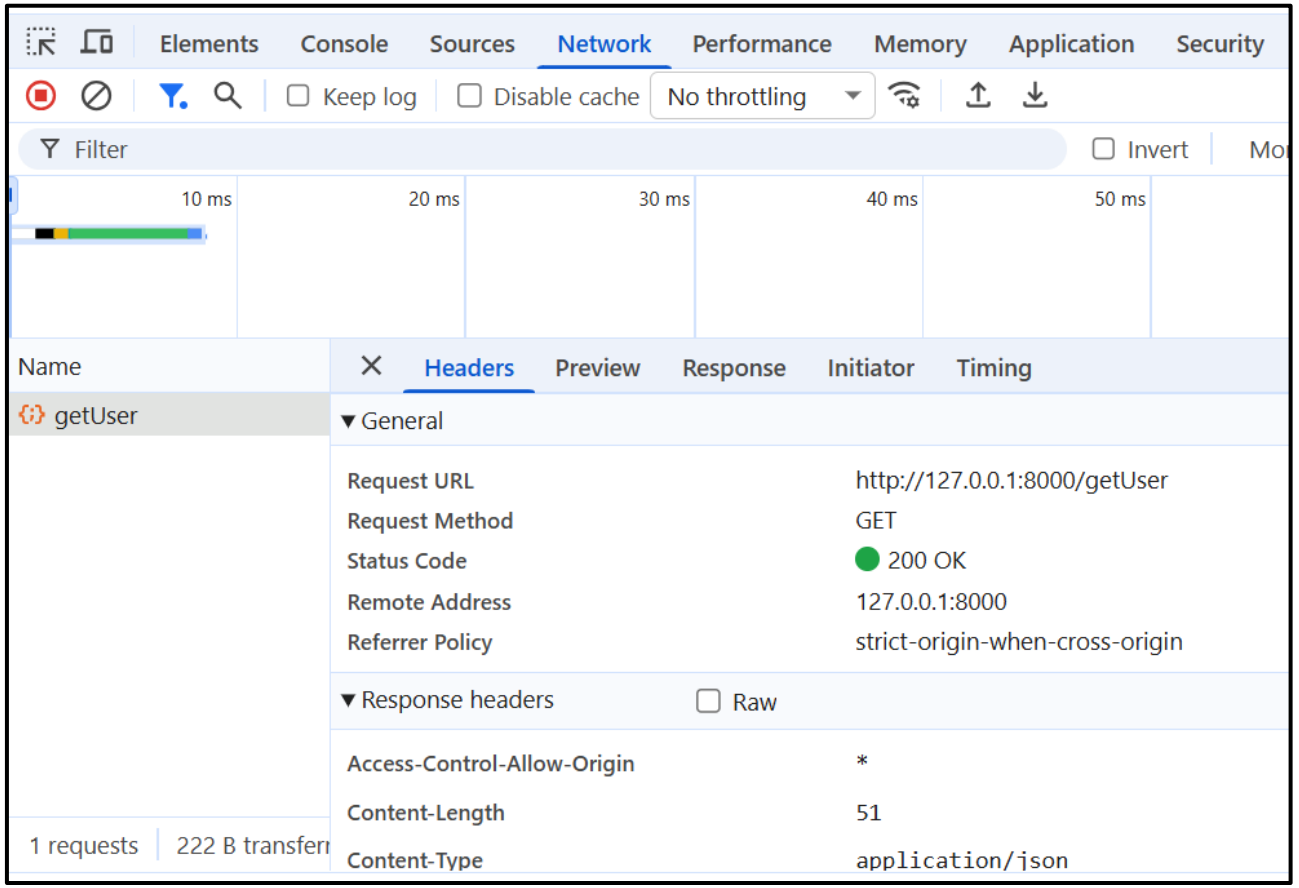


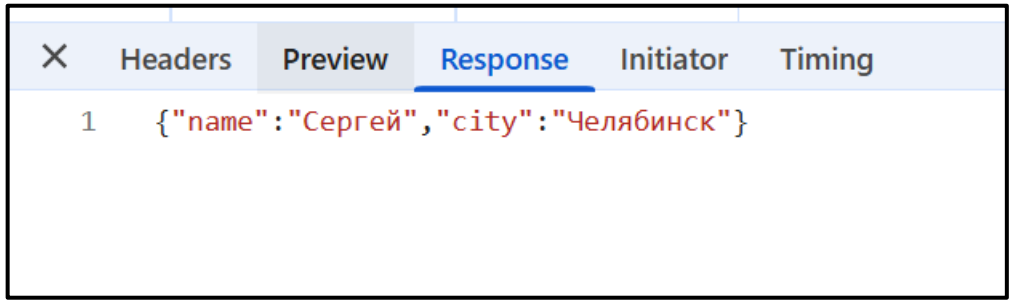
Клиентский рендеринг

НТЦ ПОЛИТЕХНИКА



- Запрос пришел в формате JSON





Южно-Уральский государственный университет

Национальный исследовательский университет

НТЦ ПОЛИТЕХНИКА

технологии • решения • интеграция

Спасибо за внимание!



2026 ООО НТЦ «Политехника»

Челябинск, ул. Карла Маркса, д. 38